Unit 4: Lists

CPTR 242 — Sequential and Parallel Data Structures and Algorithms

Walla Walla University

🤔 Arrays store elements in contiguous memory. What problems does that cause?

You have 10,000 names stored in an array. You need to insert a new name at position 0.

What has to happen? How many elements must move?

4.1 The List ADT

A **list** is an ordered sequence of elements that supports:

Operation	Description
`insert(pos, item)`	Add `item` at position `pos`
`remove(pos)`	Remove the element at `pos`
`get(pos)`	Return the element at `pos`
`find(item)`	Return the position of `item`
`size()`	Return the number of elements

4.1 The List ADT

The ADT says nothing about *how* this is implemented — that's the implementation's job.

Two classic implementations: **linked lists** and **array-based lists**.

Choosing between them comes down to which operations you need to be fast.

🤔 What if instead of storing elements next to each other, each element pointed to the next one?

Like a treasure hunt — each clue tells you where the next clue is.

What would you need to store at each "stop" in this chain? What would you need to start the hunt?

4.2 Singly-Linked Lists

A **singly-linked list** is a chain of **nodes**. Each node holds:

- A **data value**
- A **pointer to the next node**

```
head
 |
 ▼
[10 | •] ——> [20 | •] ——> [30 | •] ——> [40 | null]
```

4.2 Singly-Linked Lists

- `head` points to the first node
- The last node's `next` pointer is `null` — that's how you know you've reached the end
- There is no direct path backward — only forward

```
struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};
```

🤔 How would you search for a value in a linked list?

The list: `[10]→[20]→[30]→[40]→null`

You want to find `30`. You can only start at `head` and follow `next` pointers.

Walk through it. How many steps does it take? What is the complexity?

4.3 Singly-Linked Lists: Search and Insert

Searching — O(n)

```
Node* search(Node* head, int target) {
    Node* curr = head;
    while (curr != nullptr) {
        if (curr->data == target) return curr;
        curr = curr->next;          // advance one step
    }
    return nullptr;                 // not found
}
```

No shortcuts — must walk from `head` one node at a time. Always O(n).

Insert at Front — O(1)

```
void insertFront(Node*& head, int val) {
    Node* newNode = new Node(val);
    newNode->next = head;   // new node points to old head
    head = newNode;         // head now points to new node
}
```

Two pointer reassignments — constant time regardless of list length.

Insert After a Given Node — O(1)

Once you have a pointer to the node *before* the insertion point, inserting is O(1).

```
Before:  [10 | •]──▶[30 | •]──▶[40 | null]
Insert 20 after 10:
After:   [10 | •]──▶[20 | •]──▶[30 | •]──▶[40 | null]
```

```
void insertAfter(Node* prev, int val) {
    Node* newNode = new Node(val);
    newNode->next = prev->next;   // step 1: new node points forward
    prev->next = newNode;         // step 2: prev now points to new node
}
```

Order matters — if you did step 2 first, you'd lose the reference to the rest of the list.

🤔 How would you remove a node from a singly-linked list?

The list: `[10]→[20]→[30]→[40]→null`

You want to remove `20`. You cannot go backward — only forward.

What pointer do you need to update? Which node do you need to reach to update it?

4.4 Singly-Linked Lists: Remove

To remove a node, you need a pointer to the node **before** it — so you can redirect `prev->next` around the target.

```
Before:  [10 | •] ──> [20 | •] ──> [30 | •] ──> [40 | null]
Remove 20:
After:   [10 | •] ──────────────>  [30 | •] ──> [40 | null]
                    (20 is deallocated)
```

```
void removeAfter(Node* prev) {
    if (prev->next == nullptr) return;     // nothing to remove
    Node* toDelete = prev->next;
    prev->next = toDelete->next;           // bypass the target node!
    delete toDelete;                       // free the memory
}
```

🤔 **Are we missing an edge case for deletion?**

4.4 Singly-Linked Lists: Remove

Removing the **head** node is a special case:

```
void removeFront(Node*& head) {
    if (head == nullptr) return;
    Node* toDelete = head;
    head = head->next;
    delete toDelete;
}
```

🤔 **Singly-linked lists can only go forward. What operations become painful because of this?**

Think about: removing the *last* node. Searching backward. Finding the node just *before* a given node.

🤔 **What extra pointer could you add to each node to fix these problems?**

4.6 Doubly-Linked Lists

A **doubly-linked list** adds a `prev` pointer to every node:

```
null <—— [10 | •|•] ⇄ [20 | •|•] ⇄ [30 | •|•] ⇄ [40 | •|•] ——> null
          ▲
         head
                                                        ▲
                                                       tail
```

```
struct Node {
    int data;
    Node* next;
    Node* prev;
    Node(int val) : data(val), next(nullptr), prev(nullptr) {}
};
```

4.6 Doubly-Linked Lists

Now you can traverse in both directions and reach any node's predecessor in O(1) if you already have a pointer to it.

> Tradeoff: every operation must maintain **two pointers** per node instead of one.

More bookkeeping, but much more flexible.

4.7 Doubly-LLs: Search and Insert

Search — still O(n)

Insert Before a Given Node — O(1)

```
void insertBefore(Node* node, int val) {
    Node* newnode = new Node(val);
    newnode->next = node;
    newnode->prev = node->prev;
    if (node->prev) node->prev->next = newnode;   // link predecessor forward
    node->prev = newnode;                         // link node backward
}
```

Four pointer assignments — but all O(1). The key insight: with `prev` pointers you no longer need to find the predecessor by traversal.

🤔 Removing a node from a doubly-linked list is easier than from a singly-linked list. Why?

In a singly-linked list you needed a pointer to the node *before* the target.

In a doubly-linked list, if you have a pointer directly to the node you want to remove — do you still need to find its predecessor?

4.8 Doubly-Linked Lists: Remove

With `prev` pointers, you can remove any node in O(1) when given a direct pointer to it!

```
void removeNode(Node* node, Node*& head) {
    if (node->prev)                      // connect predecessor forward
        node->prev->next = node->next;
    else
        head = node->next;               // removing the head

    if (node->next)                      // connect successor backward
        node->next->prev = node->prev;

    delete node;
}
```

No traversal needed — four pointer reassignments and done.

Singly-linked lists require walking through the list first to find `prev`.

🤔 How do you visit every node in a linked list?

Unlike an array, you can't write `for (int i = 0; i < n; i++)` and jump to any position.

What does a loop over a linked list look like? What variable do you need? What is the stopping condition?

4.10 Linked List Traversal

The standard pattern — a pointer that walks from `head` to `null`:

```
// Print all values
void traverse(Node* head) {
    Node* curr = head;
    while (curr != nullptr) {
        std::cout << curr->data << " ";
        curr = curr->next;          // advance — never forget this line!
    }
}

// Count nodes
int length(Node* head) {
    int count = 0;
    for (Node* c = head; c != nullptr; c = c->next)
        count++;
    return count;                   // O(n) — must visit every node
}
```

`length()` is O(n) for linked lists but O(1) for `std::vector` (which stores size as a field).

🤔 Can you sort a linked list?

You know merge sort and quicksort. But they all assume you can jump to `arr[mid]` or swap arbitrary positions.

Which of those sorting algorithms would still work on a linked list? Which ones depend on random access?

4.11 Sorting Linked Lists

Merge sort is the natural choice for linked lists.

- Split into two halves using the fast/slow pointer technique
- Recursively sort each half
- Merge by relinking pointers, no extra space allocation needed

Why not quicksort? Picking a pivot and partitioning requires jumping to arbitrary positions which is O(n) per jump on a linked list, making the total $O(n^2)$ per partition level.

Merge sort works perfectly on linked lists because it only needs sequential access. Splitting and merging are both O(n) operations that naturally suit the link structure.

4.13 Linked List Dummy Nodes

Dummy nodes (also called sentinel nodes) eliminate special-case logic for empty lists and boundary conditions.

> Instead of checking `if (head == nullptr)` everywhere, add a permanent dummy node at the front whose `data` is ignored.

Dummy nodes are a classic technique to simplify code at the cost of one extra node. `std::list` uses them internally.

🤔 Can a linked list function call itself?

We used recursion for merge sort on a linked list. But can you write a recursive function that traverses or processes a list by calling itself on a smaller version?

What is the base case for a recursive linked list function? What is the recursive case?

4.14 Linked Lists and Recursion

A linked list is naturally recursive: a non-empty list is a **head node** followed by a **smaller linked list**.

```
// Recursive print — processes head, then recurses on the rest
void printRecursive(Node* node) {
    if (node == nullptr) return;        // base case: empty list
    std::cout << node->data << " ";
    printRecursive(node->next);         // recursive case: rest of list
}

// Recursive length
int lengthRec(Node* node) {
    if (node == nullptr) return 0;      // base case
    return 1 + lengthRec(node->next);  // 1 + length of the rest
}

// Recursive sum
int sumRec(Node* node) {
    if (node == nullptr) return 0;
    return node->data + sumRec(node->next);
}
```

Each function sees a list, handles the first node, and trusts the recursive call to handle the rest. This is the same mental model as recursive merge sort.

🤔 `std::list` in C++ is a doubly-linked list. But there's also `std::vector`. When would you actually choose `std::list`?

Think about: what operations are fast on each? Where does each one hurt?

Name one scenario where `std::list` clearly wins and one where `std::vector` clearly wins.

Operation	std::list	std::vector
Access element by index	O(n)	**O(1)**
Insert/remove at front	**O(1)**	O(n) shift all elements
Insert/remove at back	**O(1)**	**O(1) amortized**
Insert/remove in middle	**O(1)***	O(n) shift half the elements
Search for a value	O(n)	O(n)
Memory overhead	Higher, two pointers per node	Low, just the array
Cache performance	Poor because nodes scattered in memory	**Excellent** because contiguous

*Given a pointer to the insertion point.

In practice, `std::vector` wins for most workloads because cache locality dominates. Linked lists shine when insertions and deletions at known positions vastly outnumber random access.

🤔 What if the last node in a linked list pointed back to the first node instead of null?

The list forms a loop.

`head→[10]→[20]→[30]→[40]→head`

What new operations become easy? What becomes harder or dangerous?

4.16 Circular Lists

A **circular list** has no `null` terminator — the last node's `next` points back to the first.

```
    ┌──────────────────────────────┐
    ▼                              │
  [10] ──> [20]──>[30]──>[40]──┘
```

```
// Traversal must count steps — not check for null
void traverse(Node* head, int n) {
    Node* curr = head;
    for (int i = 0; i < n; i++) {
        std::cout << curr->data << " ";
        curr = curr->next;
    }
}
```

4.16 Circular Lists

When circular lists shine:

- **Round-robin scheduling**: cycle through processes endlessly
- **Music playlists on repeat**: wrap seamlessly from last track to first
- **Buffering**: fixed-size circular buffers (used in audio, networking)

The danger: a naive `while (curr != nullptr)` loop never terminates. Traversal requires either counting steps or checking `curr != head`.

4.17 Array-Based Lists

An **array-based list** stores elements in a contiguous block of memory and tracks the logical size separately:

```
Internal array:  [10][20][30][40][--][--][--][--]
                  0   1   2   3                  capacity=8
Logical size = 4
```

Insert at end — O(1) amortized: `arr[size++] = val`

Insert at position i — O(n): everything from index `i` onward must shift right one position

Access by index — O(1): `arr[i]` is a direct memory address

The capacity problem: what happens when `size == capacity`? The array must **grow**, typically by doubling its capacity, copying all elements to a new larger array. This is the hidden O(n) cost of `push_back` that occasionally occurs.

Why does `std::vector::push_back` claim to be O(1) "amortized" if it sometimes has to copy everything?

A single `push_back` occasionally costs O(n) when the array grows. But we say the *average* cost across many calls is O(1).

Amortized O(1): The Doubling Argument

Every time the array is full, it doubles in capacity and copies all existing elements.

Total copy work for n insertions:

```
Capacity 1 → 2:   copy 1  element
Capacity 2 → 4:   copy 2  elements
Capacity 4 → 8:   copy 4  elements
...
Capacity n/2 → n: copy n/2 elements

Total copies = 1 + 2 + 4 + ... + n/2 = n - 1
```

n insertions cause fewer than n total copy operations.

> This is why `std::vector::push_back` is described as amortized O(1). Any individual push_back might trigger an O(n) resize but that cost is spread so thinly across all the cheap insertions that the average is constant.

LAB: https://github.com/abuach/242students/blob/main/labs/lab3.md